Task 1. Express.js with Pug Template Engine

a) Set up an Express.js web application with Pug template engine.
b) Implement dynamic content rendering using Pug
c) Database integration using SQLite in Express.js

Please refer to the code in the folder **'Task 1-4'**.
The screenshots below show,

- The express.js setup done and the port is set to 3000.
- The Pug is set as the view Engine.
- The SQLite is set as database. The database name is 'students'. It contains a table 'students'
- 'viewAllStudents' selects all the records from 'students' table in 'students' database and the records are passed to 'viewAllStudents.pug' view to display the list.

App.js file:

app.js U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > app.js > ...

```
1   const express = require('express');
2   const app = express();
3   const port = 3000;
4
5   app.set('view engine', 'pug');
6
7   app.use(express.urlencoded({ extended: true }));
8
9   const sqlite3 = require('sqlite3').verbose()
10  const db = new sqlite3.Database('./db/students.db')
11
12
13  app.get('/', function (req, res) {
14      res.render('index');
15  });
16
17
18  app.get('/viewAllStudents', function (req, res) {
19
20      db.all('SELECT * FROM students ORDER BY student_id ASC', (err, rows) => {
21          res.render('viewAllStudents', { students: rows });
22      })
23  });
```

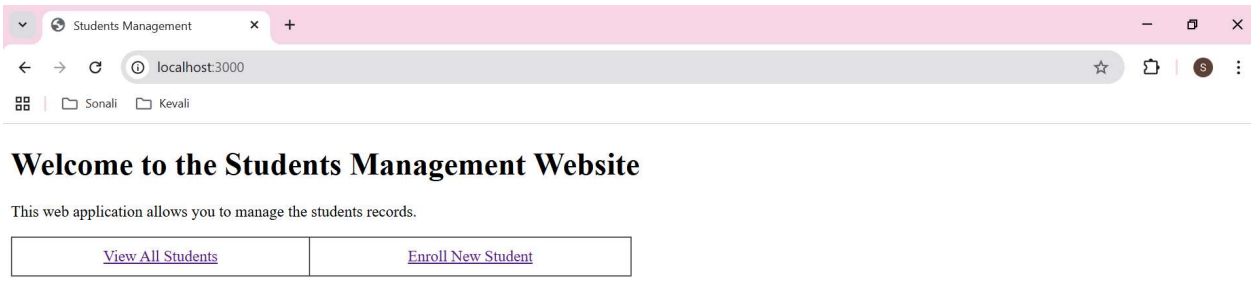
```
100
101  app.listen(port, function () {
102      console.log(`Express app listening on port ${port}!`);
103  });
104
```

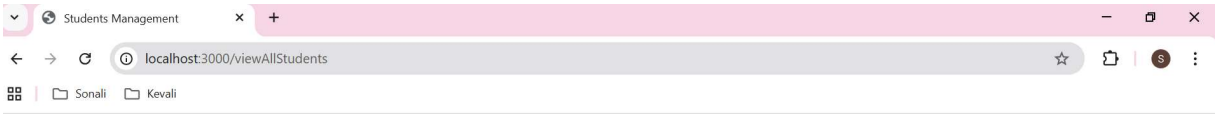
viewAllStudents.pug:

viewAllStudents.pug U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > views > viewAllStudents.pug

```
3   html
4     head
6     body
7       h1 Students Management - View All Students
8       p
9           a(href="/") Home
10      p
11          table(width="100%" border="1" cellpadding="10" cellspacing="0")
12              tbody
13                  tr
14                      th Student ID
15                      th First Name
16                      th Middle Name
17                      th Last Name
18                      th Age
19                      th(width="400px") Address
20                      th Hobbies
21                      th Action
22                  each student in students
23                      tr
24                          td #{student.student_id}
25                          td #{student.first_name}
26                          td #{student.middle_name}
27                          td #{student.last_name}
28                          td #{student.age}
29                          td #{student.address}
30                          td #{student.hobbies}
31                          td
32                              a(href=`/updateStudentInfo/${student.student_id}`) Update |
33                              a(href=`/deleteStudentRecord/${student.student_id}`) Delete
34
```





Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
5	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete
6	James	Alexander	Wilson	27	222, Walnut Blvd	Swimming, Playing Chess	Update | Delete
7	Pooja	Rahul	Mehta	2	111, Simei Drive	Programming, Swimming, Reading	Update | Delete
8	Sameer	Abdul	Menon	23	12, Choa Chu Kang Avenue 3	Cricket	Update | Delete
9	Test FName	Test MName	Test LNAme	21	Test Address	Test Hobbies	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete

Task 2. Flask with Jinja2 Template Engine

a) Set up a Flask web application with Jinja2

b) Dynamic content rendering with Jinja2

c) Handle different HTTP methods in Flask

Please refer to the code in the folder **'Task 2-4'**. The screenshots below show,

- The Flask setup is done and the required import like render_template etc is done.
- The HTTP methods like 'GET' is used for retrieving the data, 'POST" is used for update and delete records.
- The SQLite is set as database. The database name is 'students'. It contains a table 'students'
- Endpoint '/viewAllStudents' selects all the records from 'students' table in 'students' database and the records are passed to 'viewAllStudents.html' template to display the list.

App.py:

app.py U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > app.py > ...

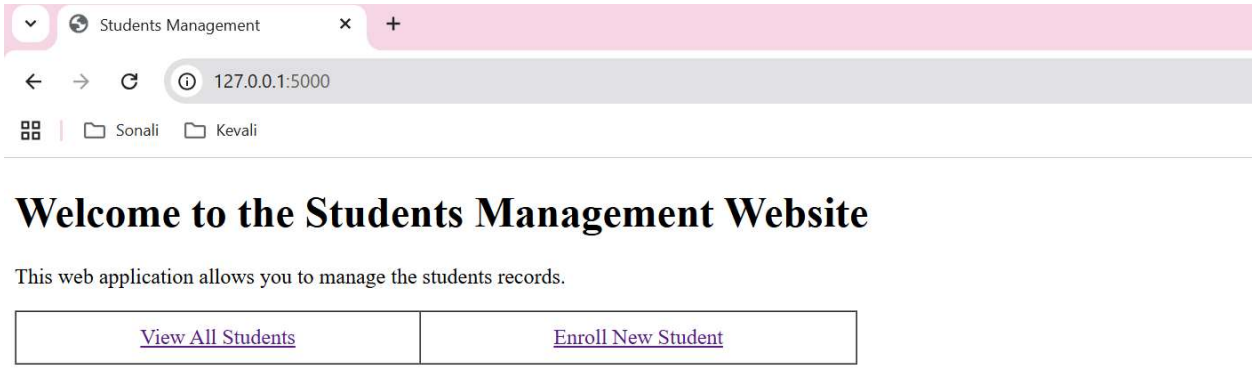
```python
from flask import Flask
from flask import render_template
from flask import request
import sqlite3

app = Flask(__name__)

@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

@app.route('/viewAllStudents', methods=['GET'])
def viewAllStudents():

    conn = sqlite3.connect('./db/students.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM students ORDER BY student_id ASC')
    students = cursor.fetchall()
    conn.close()

    return render_template('viewAllStudents.html', students=students)


@app.route('/enrollNewStudent', methods=['GET', 'POST'])
def enrollNewStudent():

    if request.method == 'GET':

        return render_template('enrollNewStudent.html')

    elif request.method == 'POST':

        first_name = request.form['first_name']
        middle_name = request.form['middle_name']
        last_name = request.form['last_name']
        age = request.form['age']
        address = request.form['address']
        hobbies = request.form['hobbies']

        conn = sqlite3.connect('./db/students.db')
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO students (first_name, middle_name, last_name, age, address, hobbies)
            VALUES (?, ?, ?, ?, ?, ?)
        ''', (first_name, middle_name, last_name, age, address, hobbies))
        conn.commit()
        conn.close()

        return 'A new student has been enrolled with Student ID: {} <a href="/">Home</a>'.format(cursor.lastrowid)
```

viewAllStudents.html:

viewAllStudents.html U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > templates > viewAllStudents.html > ...

```
<!DOCTYPE html>
<html>

    <head>
        <title>Students Management</title>
    </head>

    <body>
        <h1>Students Management - View All Students</h1>
        <p><a href="/">Home</a></p>
        <p>
            <table width="100%" border="1" cellpadding="10" cellspacing="0">
                <tbody>
                    <tr>
                        <th>Student ID</th>
                        <th>First Name</th>
                        <th>Middle Name</th>
                        <th>Last Name</th>
                        <th>Age</th>
                        <th width="400px">Address</th>
                        <th>Hobbies</th>
                        <th>Action</th>
                    </tr>
                    {% for student in students %}
                        <tr>
                            <td>{{ student[0] }}</td>
                            <td>{{ student[1] }}</td>
                            <td>{{ student[2] }}</td>
                            <td>{{ student[3] }}</td>
                            <td>{{ student[4] }}</td>
                            <td>{{ student[5] }}</td>
                            <td>{{ student[6] }}</td>
                            <td><a href="/updateStudentInfo/{{ student[0] }}">Update</a> | <a href="/deleteStudentRecord/{{ student[0] }}">Delete</a></td>
                        </tr>
                    {% endfor %}
                </tbody>
            </table>
        </p>
    </body>

</html>
```



Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
6	Test 1 FName	Test 1 MName	Test 1 LName	35	Test Address, USA	Singing	Update | Delete
7	Test 2 FName	Test 2 MName	Test 2 LName	35	Test Address, USA	Singing	Update | Delete
8	Test 3 FName	Test 3 MName	Test 3 LName	35	Test Address, USA	Singing	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete
11	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete

Task 3. Static Assets and Session Management

a) Serve static assets in Express.js

Please refer to the code in folder **'Task 3 - Express'.**

- Below line means any request to /images will retrieve the files from images/ folder located at the root.

```
app.use('/images', express.static(process.cwd() + '/images'));
```

- The images can be used in 'index.pug' view as below.

```
img(src='./images/Image1.jpg', style='width: 600px;')
```

- Screenshot for code for 'app.js' is on the next page.
- Screenshot for code for 'index.pug' is on the next page.

b) Manage session state using Express.js sessions

Please refer to the code in folder **'Task 3 - Express'.**

In app.js,

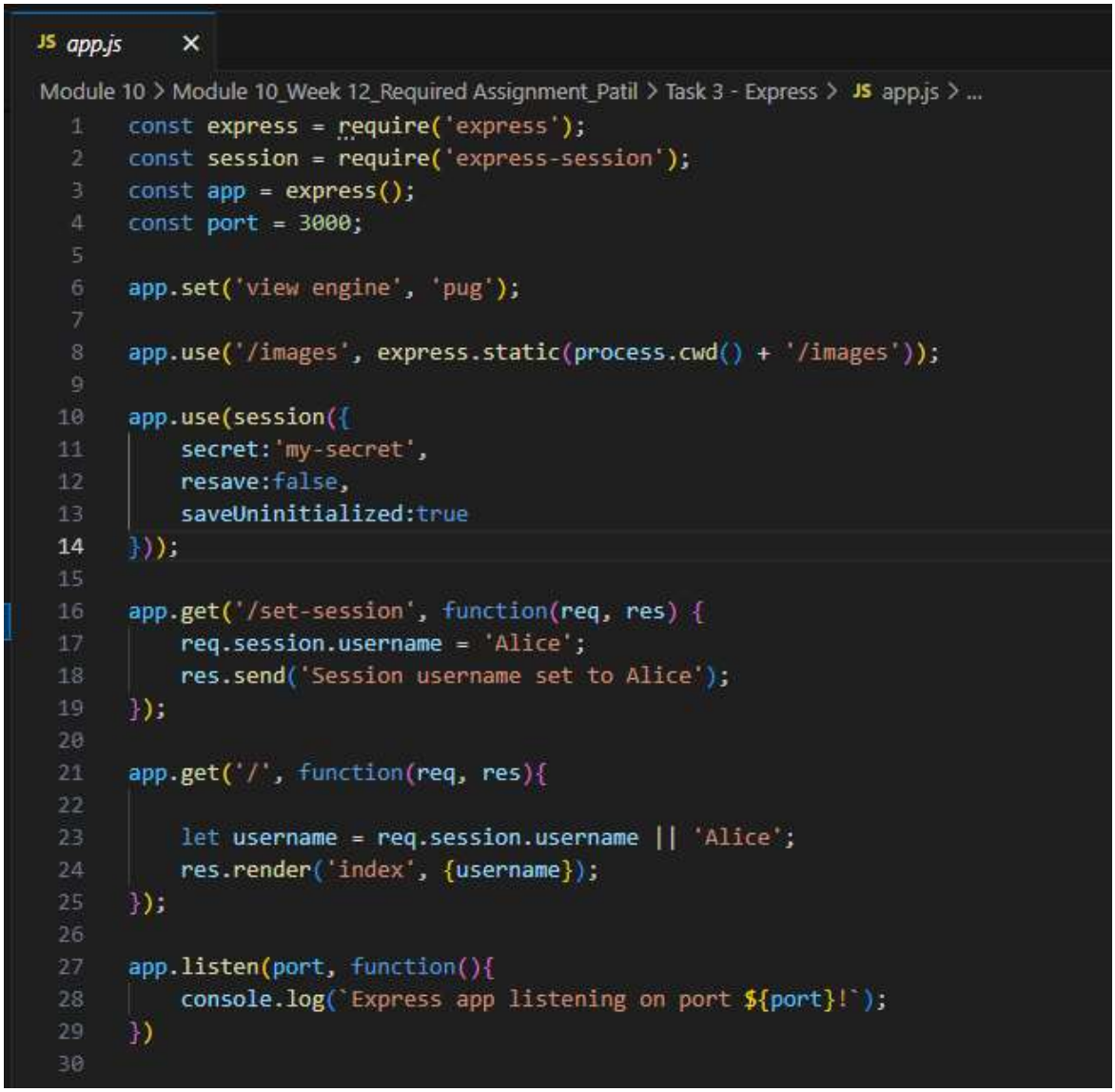
- The express.js setup done and the port is set to 3000. 'express-session' is used for session management.
- 'express-session' is configured with a secret and standard options.
- Route '/set-session' sets a session variable username to 'Alice'.

- Route '/', checks req.session.username value, defaults to 'Alice' if not set and passes it to the index.pug template.

In 'Index.pug'

- 'Index.pug' view checks if 'username' has value. If yes, it displays it in the greeting (Hi Alice!). Else it just displays greeting without username.

Code for 'app.js' as below.



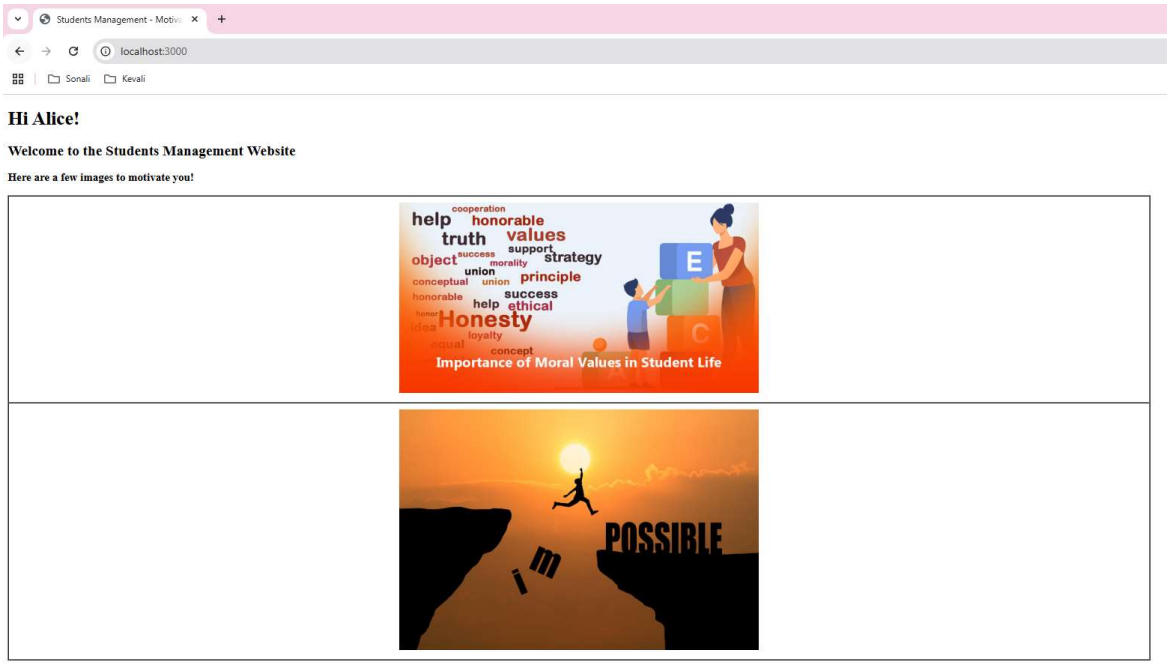
Code for ‘index.pug’ as below.

index.pug

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 3 - Express > views > index.pug

```
doctype html

html
  head
    title Students Management - Motivational Images
  body
    if username
      h1 Hi #{username}!
    else
      h1 Hi!

    h2 Welcome to the Students Management Website
    h3 Here are a few images to motivate you!

    table(width="80%" height="99%" border="1" cellpadding="10" cellspacing="0")
        tbody
            tr
                td(align="center")
                    img(src='./images/Image1.jpg', style='width: 600px;')
            tr
                td(align="center")
                    img(src='./images/Image2.jpg', style='width: 600px;')

```

The webpage displayed below:



c) Serve static assets in Flask

Please refer to the code in folder 'Task 3 - Flask.

- The images are stored in the 'static' folder located at the root folder.
- We can use those static images in the html templates as below.

```
<img src="../static/Image1.jpg" width="600px" />
```

- Similarly, we can use other static assets like stylesheet etc.
- Please find below the screenshot of 'index.html'.

index.html

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 3 - Flask > templates > index.html > html > body

```
<!DOCTYPE html>
<html>
<head>
    <title>Students Management - Motivational Images</title>
</head>
<body>
    {% if username %}
        <h1>Hi {{ username }}!</h1>
    {% else %}
        <h1>Hi!</h1>
    {% endif %}
    <h2>
        Welcome to the Students Management Website
    </h2>
    <h3>
        Here are a few images to motivate you!
    </h3>

    <table width="80%" height="99%" border="1" cellpadding="10" cellspacing="0">
        <tr>
            <td align="center">
                <img src="../static/Image1.jpg" width="600px" />
            </td>
            <td align="center">
                <img src="../static/Image2.jpg" width="600px" />
            </td>
        </tr>
    </table>

</body>
</html>
```

d) Manage session state using Flask sessions

Please refer to the code in folder 'Task 3 - Flask.

- The 'session' is imported from 'flask'.
- Route '/set-session' sets a session variable username to 'Alice'.
- Route '/', checks req.session.username value, defaults to 'Alice' if not set and passes it to the index.html template.

In ‘Index.html

- ‘Index.html template checks if ‘username’ has value. If yes, it displays it in the greeting (Hi Alice!). Else it just displays greeting without username.

Screenshot for code in app.py:

app.py

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 3 - Flask > app.py > ...

```python
from flask import Flask, session
from flask import render_template

app = Flask(__name__)
app.secret_key = 'my-secret'

@app.route('/set-session')
def set_session():
    session['username'] = 'Alice'
    return 'Session username set to Alice'

@app.route('/')
def index():
    username = session.get('username', 'Alice')
    return render_template('index.html', username=username)

```

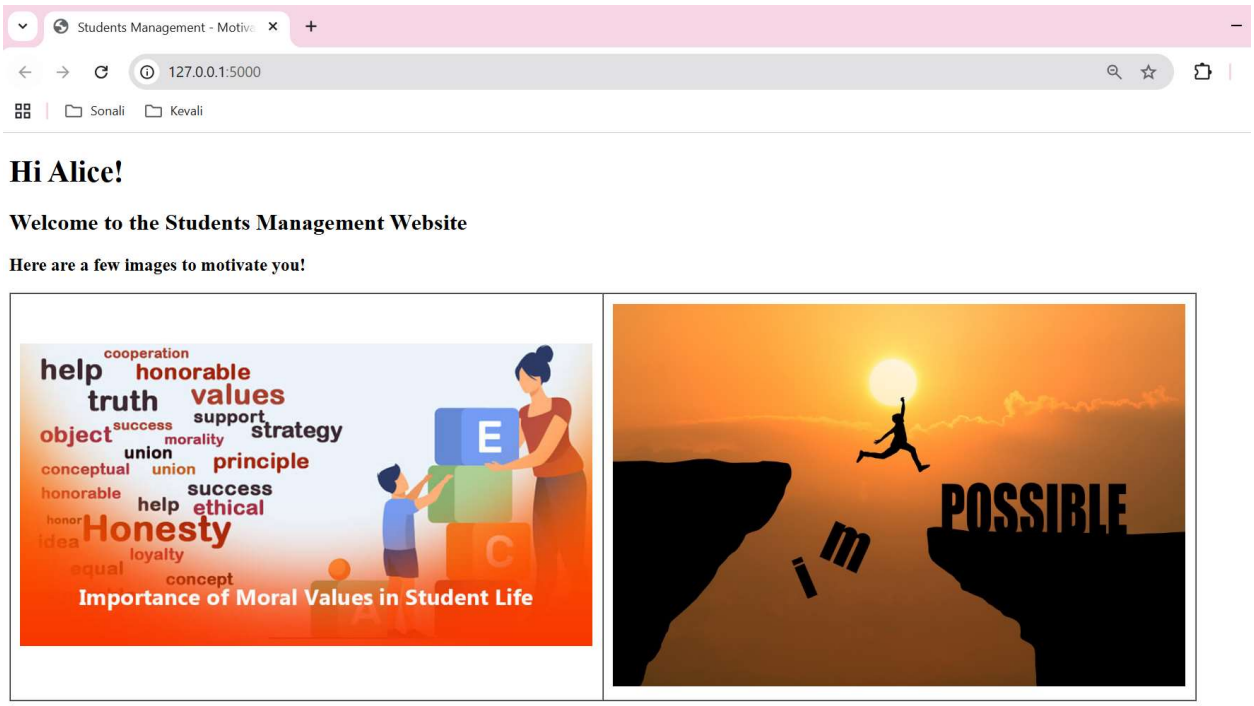
Screenshot for code in index.html:

index.html

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 3 - Flask > templates > index.html > html > body

```html
<!DOCTYPE html>
<html>
<head>
    <title>Students Management - Motivational Images</title>
</head>
<body>
    {% if username %}
        <h1>Hi {{ username }}!</h1>
    {% else %}
        <h1>Hi!</h1>
    {% endif %}
    <h2>
        Welcome to the Students Management Website
    </h2>
    <h3>
        Here are a few images to motivate you!
    </h3>

    <table width="80%" height="99%" border="1" cellpadding="10" cellspacing="0">
        <tr>
            <td align="center">
                <img src="../static/Image1.jpg" width="600px" />
            </td>
            <td align="center">
                <img src="../static/Image2.jpg" width="600px" />
            </td>
        </tr>
    </table>

</body>
</html>
```

The webpage displays below:



Task 4. Database interaction and CRUD operations

a) Set up a Flask application to perform CRUD operations with SQLite.

Please refer to the code in folder **'Task 2 – 4'**.

- Flask is setup and render_template, request are imported.
- Sqlite3 is setup as database.
- A GET endpoint is set to the root path '/'. It renders Jinja2 template 'index.html'.

Code from app.py:

app.py U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > app.py > viewAllStudents

```python
from flask import Flask
from flask import render_template
from flask import request
import sqlite3

app = Flask(__name__)

@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

```

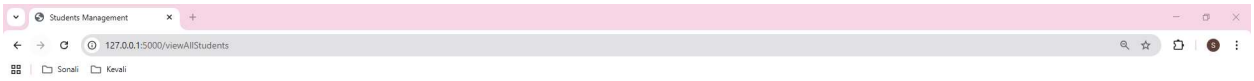
Code from index.html:

index.html U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > templates > index.html > ...

```html
<!DOCTYPE html>
<html>

    <head>
        <title>Students Management</title>
    </head>

    <body>
        <h1>Welcome to the Students Management Website</h1>
        <p>This web application allows you to manage the students records. </p>
        <table width="50%" height="50%" border="1" cellpadding="10" cellspacing="0">
            <tbody>
                <tr>
                    <td align="center"><a href="/viewAllStudents">View All Students</a></td>
                    <td align="center"><a href="/enrollNewStudent">Enroll New Student</a></td>
                </tr>
            </tbody>
        </table>
    </body>

</html>

```

- A GET endpoint '/viewAllStudents' connects to sqlite database ‘students’ and fetches all the rows from ‘students’ table order by student_id. Jinja2 template ‘viewAllStudents.html’ renders the list of all the retrieved students’ records.



Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
6	Test 1 FName	Test 1 MName	Test 1 LName	35	Test Address, USA	Singing	Update | Delete
7	Test 2 FName	Test 2 MName	Test 2 LName	35	Test Address, USA	Singing	Update | Delete
8	Test 3 FName	Test 3 MName	Test 3 LName	35	Test Address, USA	Singing	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete
11	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete

Code from app.py:

```python

@app.route('/viewAllStudents', methods=['GET'])
def viewAllStudents():

    conn = sqlite3.connect('./db/students.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM students ORDER BY student_id ASC')
    students = cursor.fetchall()
    conn.close()

    return render_template('viewAllStudents.html', students=students)


```

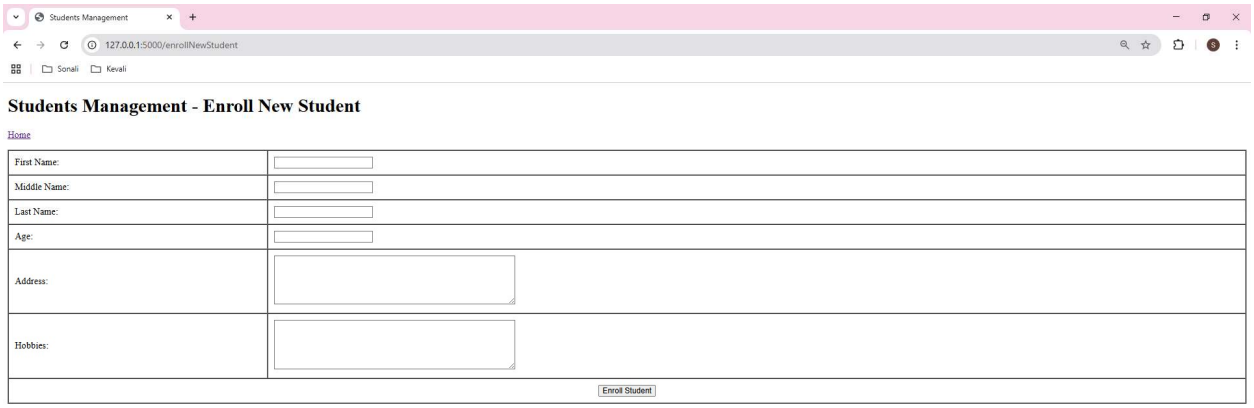
Code from viewAllStudents.html:

viewAllStudents.html U

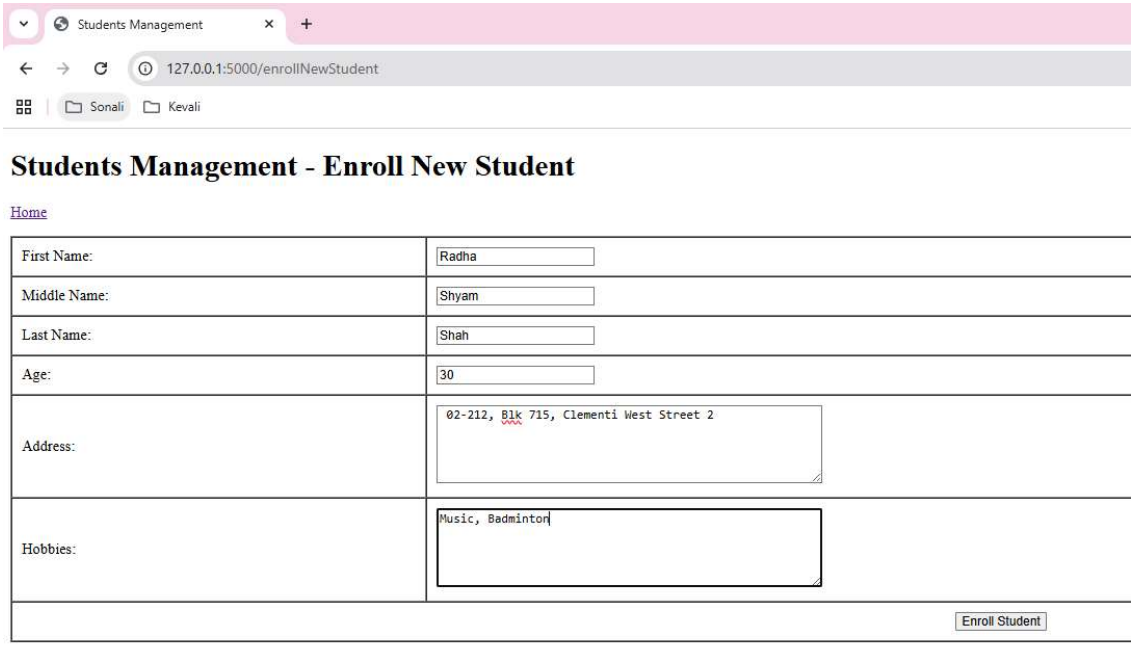
Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > templates > viewAllStudents.html > html

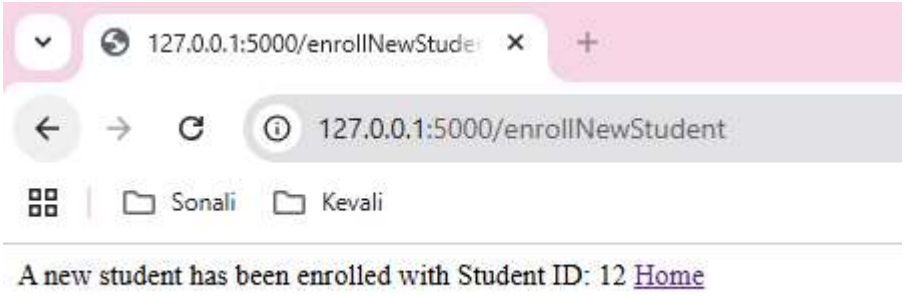
```html
<!DOCTYPE html>
<html>

    <head>
        <title>Students Management</title>
    </head>

    <body>
        <h1>Students Management - View All Students</h1>
        <p><a href="/">Home</a></p>
        <p>
            <table width="100%" border="1" cellpadding="10" cellspacing="0">
                <tbody>
                    <tr>
                        <th>Student ID</th>
                        <th>First Name</th>
                        <th>Middle Name</th>
                        <th>Last Name</th>
                        <th>Age</th>
                        <th width="400px">Address</th>
                        <th>Hobbies</th>
                        <th>Action</th>
                    </tr>
                    {% for student in students %}
                        <tr>
                            <td>{{ student[0] }}</td>
                            <td>{{ student[1] }}</td>
                            <td>{{ student[2] }}</td>
                            <td>{{ student[3] }}</td>
                            <td>{{ student[4] }}</td>
                            <td>{{ student[5] }}</td>
                            <td>{{ student[6] }}</td>
                            <td><a href="/updateStudentInfo/{{ student[0] }}">Update</a> | <a href="/deleteStudentRecord/{{ student[0] }}">Delete</a></td>
                        </tr>
                    {% endfor %}
                </tbody>
            </table>
        </p>
    </body>

</html>
```

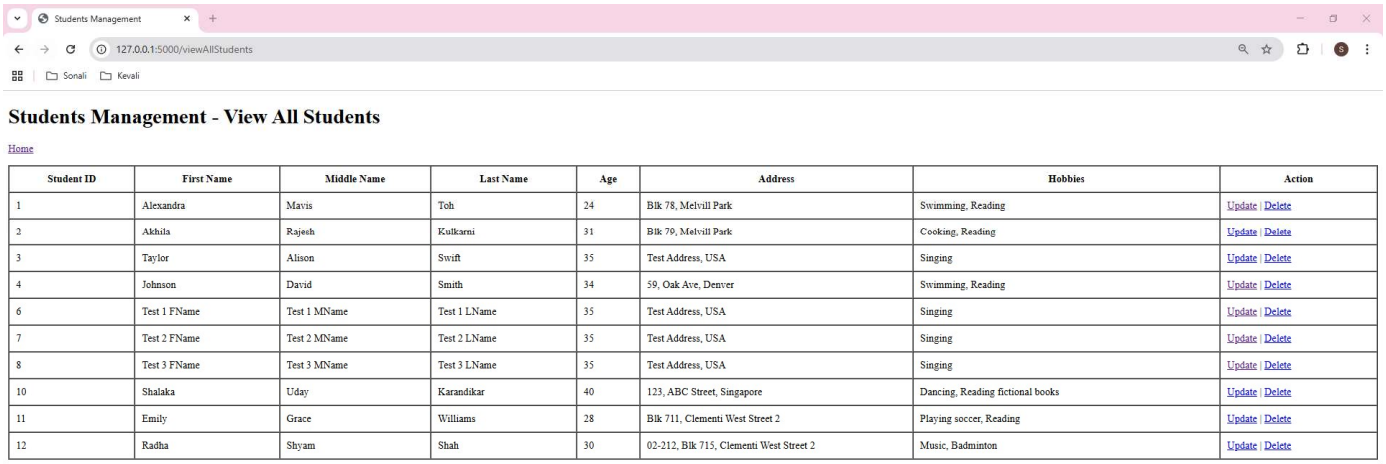
- A GET endpoint ‘/enrollNewStudent’ renders Jinja2 template ‘enrollNewStudent.html’. It displays an empty form for enrolling the student.



- A POST endpoint '/enrollNewStudent' retrieves the data entered by the user in 'enrollNewStudent.html' template and inserts a new student record in students database table. It provides a link for user to redirect to Home page.







Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
6	Test 1 FName	Test 1 MName	Test 1 LName	35	Test Address, USA	Singing	Update | Delete
7	Test 2 FName	Test 2 MName	Test 2 LName	35	Test Address, USA	Singing	Update | Delete
8	Test 3 FName	Test 3 MName	Test 3 LName	35	Test Address, USA	Singing	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete
11	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete
12	Radha	Shyam	Shah	30	02-212, Blk 715, Clementi West Street 2	Music, Badminton	Update | Delete

Code from app.py:

```
22
23
24  @app.route('/enrollNewStudent', methods=['GET', 'POST'])
25  def enrollNewStudent():
26
27      if request.method == 'GET':
28
29          return render_template('enrollNewStudent.html')
30
31      elif request.method == 'POST':
32
33          first_name = request.form['first_name']
34          middle_name = request.form['middle_name']
35          last_name = request.form['last_name']
36          age = request.form['age']
37          address = request.form['address']
38          hobbies = request.form['hobbies']
39
40          conn = sqlite3.connect('./db/students.db')
41          cursor = conn.cursor()
42          cursor.execute('''
43              INSERT INTO students (first_name, middle_name, last_name, age, address, hobbies)
44              VALUES (?, ?, ?, ?, ?, ?)
45          ''', (first_name, middle_name, last_name, age, address, hobbies))
46          conn.commit()
47          conn.close()
48
49          return 'A new student has been enrolled with Student ID: {} <a href="/">Home</a>'.format(cursor.lastrowid)
50
51
52
```

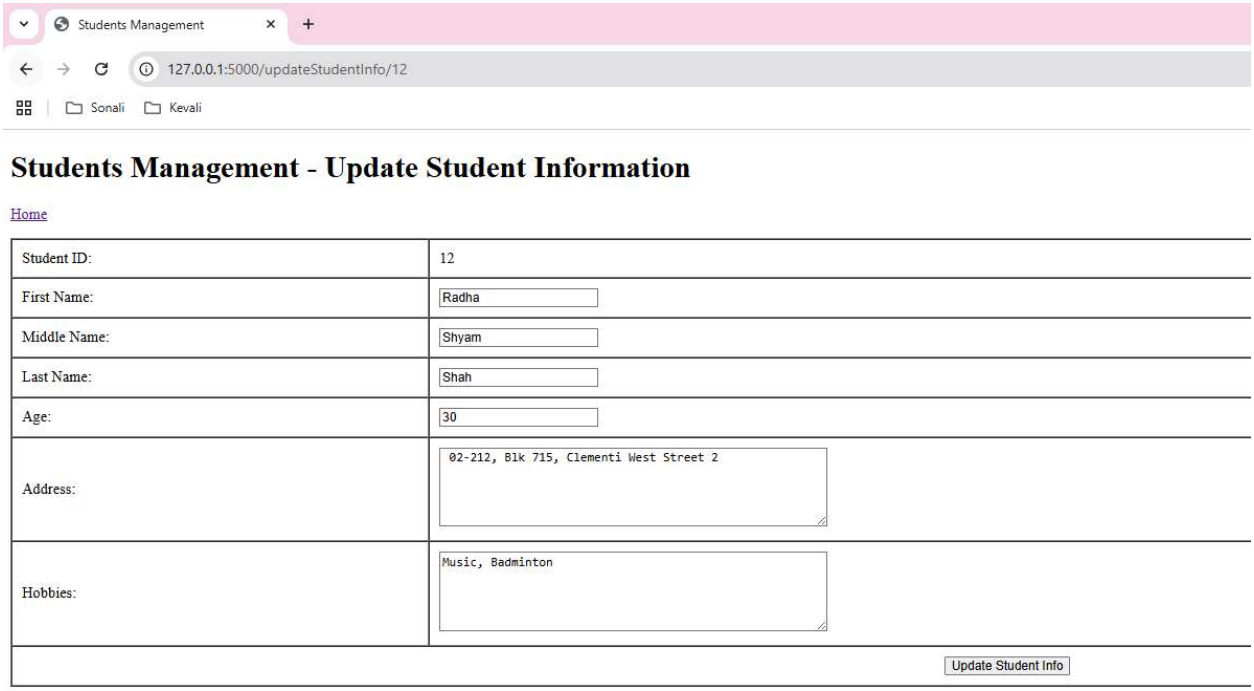
Code from enrollNewStudent.html:

enrollNewStudent.html U

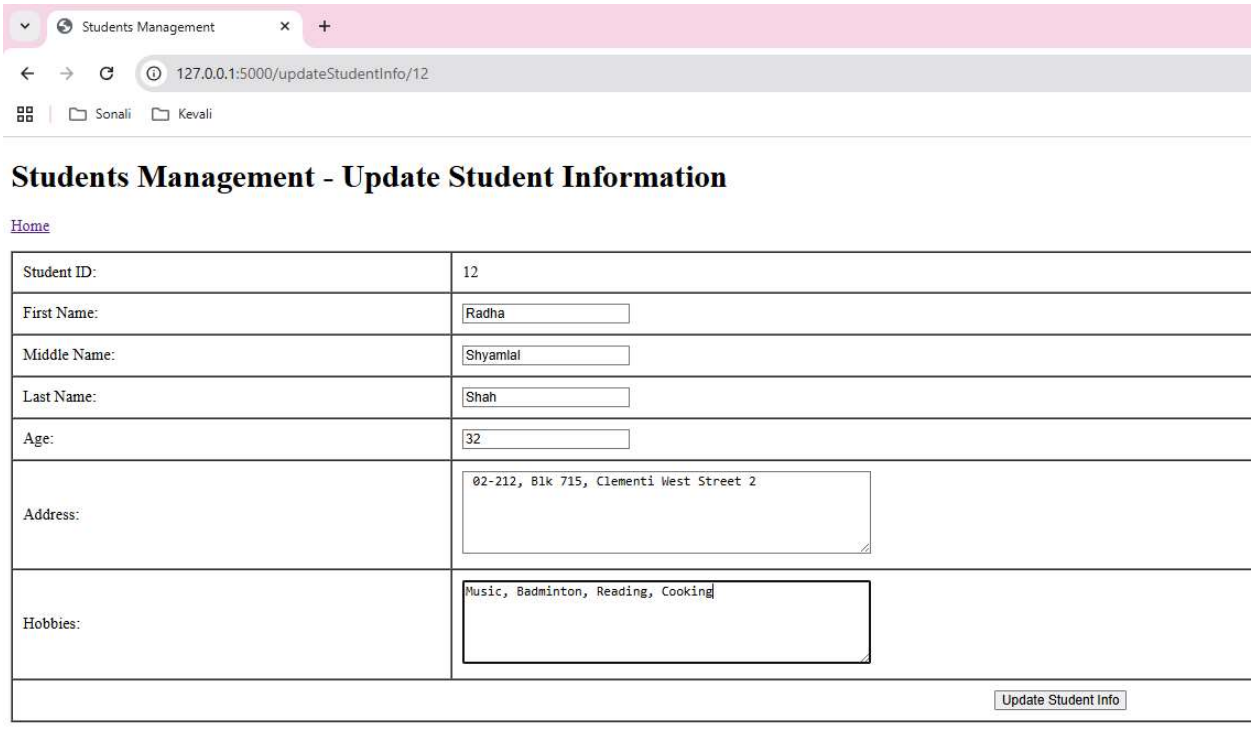
Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > templates > enrollNewStudent.html > ...

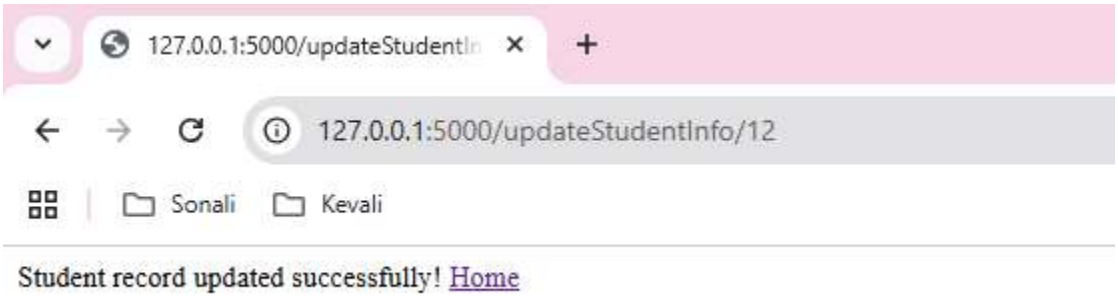
```
<!DOCTYPE html>
<html>

    <head>
        <title>Students Management</title>
    </head>

    <body>
        <h1>Students Management - Enroll New Student</h1>
        <p><a href="/">Home</a></p>
        <p>
            <form action="/enrollNewStudent" method="post">
                <table width="100%" border="1" cellpadding="10" cellspacing="0">
                    <tbody>
                        <tr>
                            <td>First Name:</td>
                            <td><input type="text" name="first_name" required="required" /></td>
                        </tr>
                        <tr>
                            <td>Middle Name:</td>
                            <td><input type="text" name="middle_name" required="required" /></td>
                        </tr>
                        <tr>
                            <td>Last Name:</td>
                            <td><input type="text" name="last_name" required="required" /></td>
                        </tr>
                        <tr>
                            <td>Age:</td>
                            <td><input type="number" name="age" required="required" /></td>
                        </tr>
                        <tr>
                            <td>Address:</td>
                            <td><textarea name="address" rows="5" cols="50"  required="required"> </textarea></td>
                        </tr>
                        <tr>
                            <td>Hobbies:</td>
                            <td><textarea name="hobbies" rows="5" cols="50" required="required"></textarea></td>
                        </tr>
                        <tr>
                            <td colspan="2" align="center"><input type="submit" value="Enroll Student" /></td>
                        </tr>
                    </tbody>
                </table>
            </form>
        </p>
    </body>

</html>

```

- A GET endpoint '/updateStudentInfo' accepts 'student_id' as parameter and retrieves the record for that particular student from the sqlite database. It then displays that record in Jinja2 template, 'enrollNewStudent.html'. User can edit the existing record data here.



- A POST endpoint '/updateStudentInfo' accepts 'student_id' as parameter, retrieves the data edited by user for that particular student id from Jinja2 template, 'enrollNewStudent.html'. It then updates the student record in sqlite students database. Then it provides a link for user to redirect to Home page.





Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
6	Test 1 FName	Test 1 MName	Test 1 LName	35	Test Address, USA	Singing	Update | Delete
7	Test 2 FName	Test 2 MName	Test 2 LName	35	Test Address, USA	Singing	Update | Delete
8	Test 3 FName	Test 3 MName	Test 3 LName	35	Test Address, USA	Singing	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete
11	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete
12	Radha	Shyamlal	Shah	32	02-212, Blk 715, Clementi West Street 2	Music, Badminton, Reading, Cooking	Update | Delete

Code from app.py:

```
51
52
53  @app.route('/updateStudentInfo/<student_id>', methods=['GET', 'POST'])
54  def updateStudentInfo(student_id):
55
56      if request.method == 'GET':
57
58          conn = sqlite3.connect('./db/students.db')
59          cursor = conn.cursor()
60          cursor.execute('SELECT * FROM students WHERE student_id = ?', (student_id,))
61          student = cursor.fetchone()
62          conn.close()
63
64          return render_template('updateStudentInfo.html', student=student)
65
66      elif request.method == 'POST':
67
68          first_name = request.form['first_name']
69          middle_name = request.form['middle_name']
70          last_name = request.form['last_name']
71          age = request.form['age']
72          address = request.form['address']
73          hobbies = request.form['hobbies']
74
75          conn = sqlite3.connect('./db/students.db')
76          cursor = conn.cursor()
77          cursor.execute('''
78              UPDATE students
79              SET first_name = ?, middle_name = ?, last_name = ?, age = ?, address = ?, hobbies = ?
80              WHERE student_id = ?
81          ''', (first_name, middle_name, last_name, age, address, hobbies, student_id))
82          conn.commit()
83          conn.close()
84
85          return 'Student record updated successfully! <a href="/">Home</a>'
86
87
88
```

Code from updateStudentInfo.html:

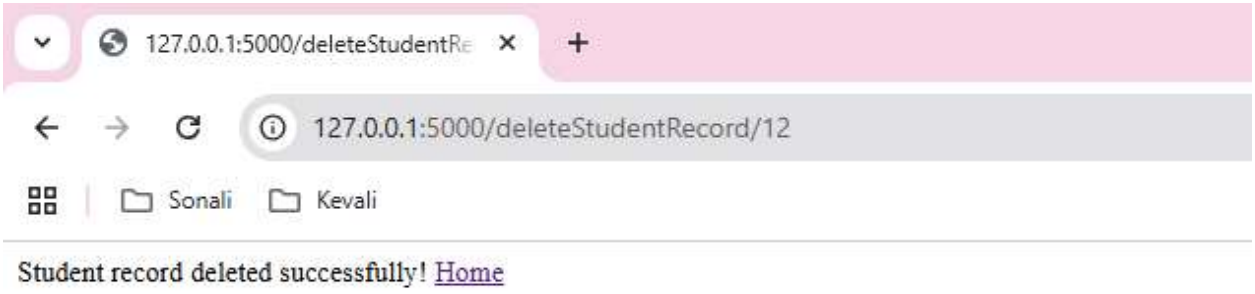
updateStudentInfo.html U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > templates > updateStudentInfo.html > html

```
<!DOCTYPE html>
<html>
    <head>
        <title>Students Management</title>
    </head>

    <body>
        <h1>Students Management - Update Student Information</h1>
        <p><a href="/">Home</a></p>
        <p>
            <form action="/updateStudentInfo/{{ student[0] }}" method="post">
                <table width="100%" border="1" cellpadding="10" cellspacing="0">
                    <tbody>
                        <tr>
                            <td>Student ID:</td>
                            <td>{{ student[0] }}</td>
                        </tr>
                        <tr>
                            <td>First Name:</td>
                            <td><input type="text" name="first_name" required="required" value="{{ student[1] }}" /></td>
                        </tr>
                        <tr>
                            <td>Middle Name:</td>
                            <td><input type="text" name="middle_name" required="required" value="{{ student[2] }}" /></td>
                        </tr>
                        <tr>
                            <td>Last Name:</td>
                            <td><input type="text" name="last_name" required="required" value="{{ student[3] }}" /></td>
                        </tr>
                        <tr>
                            <td>Age:</td>
                            <td><input type="number" name="age" required="required" value="{{ student[4] }}" /></td>
                        </tr>
                        <tr>
                            <td>Address:</td>
                            <td><textarea  name="address" rows="5" cols="50" required="required">{{ student[5] }}</textarea></td>
                        </tr>
                        <tr>
                            <td>Hobbies:</td>
                            <td> <textarea name="hobbies" rows="5" cols="50" required="required">{{ student[6] }}</textarea></td>
                        </tr>
                        <tr>
                            <td colspan="2" align="center"><input type="submit" value="Update Student Info" /></td>
                        </tr>
                    </tbody>
                </table>
            </form>
        </p>
    </body>
```

- A GET endpoint '/deleteStudentRecord' accepts 'student_id' as parameter, it then retrieves the record for that particular student from 'students' database table and displays it using 'deleteStudentRecord.html' Jinja2 template.

- A POST endpoint '/deleteStudentRecord' accepts 'student_id' as parameter, it then Deletes the record for that student by its id. Then it provides a link for user to redirect to Home page.



Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
6	Test 1 FName	Test 1 MName	Test 1 LName	35	Test Address, USA	Singing	Update | Delete
7	Test 2 FName	Test 2 MName	Test 2 LName	35	Test Address, USA	Singing	Update | Delete
8	Test 3 FName	Test 3 MName	Test 3 LName	35	Test Address, USA	Singing	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete
11	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete

Code from app.py:

```
86
87
88
89  @app.route('/deleteStudentRecord/<student_id>', methods=['GET', 'POST'])
90  def deleteStudentRecord(student_id):
91
92      if request.method == 'GET':
93
94          conn = sqlite3.connect('./db/students.db')
95          cursor = conn.cursor()
96          cursor.execute('SELECT * FROM students WHERE student_id = ?', (student_id,))
97          student = cursor.fetchone()
98          conn.close()
99
100         return render_template('deleteStudentRecord.html', student=student)
101
102     elif request.method == 'POST':
103
104         conn = sqlite3.connect('./db/students.db')
105         cursor = conn.cursor()
106         cursor.execute('DELETE FROM students WHERE student_id = ?', (student_id,))
107         conn.commit()
108         conn.close()
109
110         return 'Student record deleted successfully! <a href="/">Home</a>'
111
```

Code from deleteStudentRecord.html:

deleteStudentRecord.html U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 2-4 > templates > deleteStudentRecord.html > html > head

```
<!DOCTYPE html>
<html>

    <head>
        <title>Students Management</title>
    </head>
    <body>
        <h1>Students Management - Delete Student Record</h1>
        <p><a href="/">Home</a></p>
        <p>
            <form action="/deleteStudentRecord/{{ student[0] }}" method="post">
                <table width="100%" border="1" cellpadding="10" cellspacing="0">
                    <tbody>
                        <tr>
                            <td>Student ID:</td>
                            <td>{{ student[0] }}</td>
                        </tr>
                        <tr>
                            <td>First Name:</td>
                            <td>{{ student[1] }}</td>
                        </tr>
                        <tr>
                            <td>Middle Name:</td>
                            <td>{{ student[2] }}</td>
                        </tr>
                        <tr>
                            <td>Last Name:</td>
                            <td>{{ student[3] }} </td>
                        </tr>
                        <tr>
                            <td>Age:</td>
                            <td>{{ student[4] }}</td>
                        </tr>
                        <tr>
                            <td>Address:</td>
                            <td>{{ student[5] }}</td>
                        </tr>
                        <tr>
                            <td>Hobbies:</td>
                            <td>{{ student[6] }}</td>
                        </tr>
                        <tr>
                            <td colspan="2" align="center"><input type="submit" value="Delete Student Record"></td>
                        </tr>
                    </tbody>
                </table>
            </form>
        </p>
    </body>
```

b) Set up CRUD operations in Express.js using SQLite.

Please refer to the code in folder **'Task 1 – 4'**.

- Express.js is setup and port is set to 3000.
- The view engine is set as 'pug'.
- Sqlite3 is setup as database.
- A GET endpoint is set to the root path '/'. It renders pug view 'index.pug.

Welcome to the Students Management Website

This web application allows you to manage the students records.

View All Students	Enroll New Student

Code from app.js:

```
app.js  U
Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > app.js > ...
1   const express = require('express');
2   const app = express();
3   const port = 3000;
4
5   app.set('view engine', 'pug');
6
7   app.use(express.urlencoded({ extended: true }));
8
9   const sqlite3 = require('sqlite3').verbose()
10  const db = new sqlite3.Database('./db/students.db')
11
12
13  app.get('/', function (req, res) {
14      res.render('index');
15  });
16
17
```

```
100
101   app.listen(port, function () {
102       console.log(`Express app listening on port ${port}!`);
103   });
104
```

Code from index.pug:

index.pug U

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > views > index.pug

```
doctype html

html
  head
    title Students Management
  body
    h1 Welcome to the Students Management Website
    p This web application allows you to manage the students records.

    table(width="50%" height="50%" border="1" cellpadding="10" cellspacing="0")
        tbody
            tr
                td(align="center")
                    a(href="/viewAllStudents") View All Students
                td(align="center")
                    a(href="/enrollNewStudent") Enroll New Student
```

- A GET endpoint '/viewAllStudents' connects to sqlite database ‘students’ and fetches all the rows from ‘students’ table order by student_id. A pug view ‘viewAllStudents.pug’ renders the list of all the retrieved students’ records.

Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
5	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete
6	James	Alexander	Wilson	27	222, Walnut Blvd	Swimming, Playing Chess	Update | Delete
7	Pooja	Rahul	Mehta	2	111, Simei Drive	Programming, Swimming, Reading	Update | Delete
8	Sameer	Abdul	Menon	23	12, Choa Chu Kang Avenue 3	Cricket	Update | Delete
9	Test FName	Test MName	Test LNAme	21	Test Address	Test Hobbies	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete

Code from app.js:

```
app.get('/viewAllStudents', function (req, res) {

    db.all('SELECT * FROM students ORDER BY student_id ASC', (err, rows) => {
        res.render('viewAllStudents', { students: rows });
    })
});
```

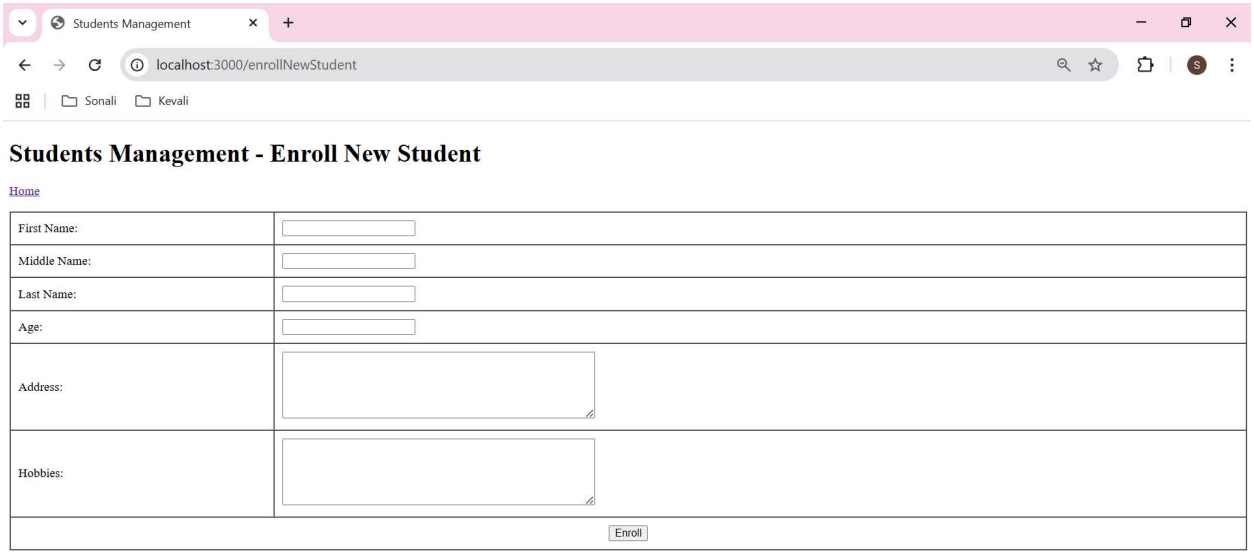
Code from viewAllStudents.pug:

viewAllStudents.pug U

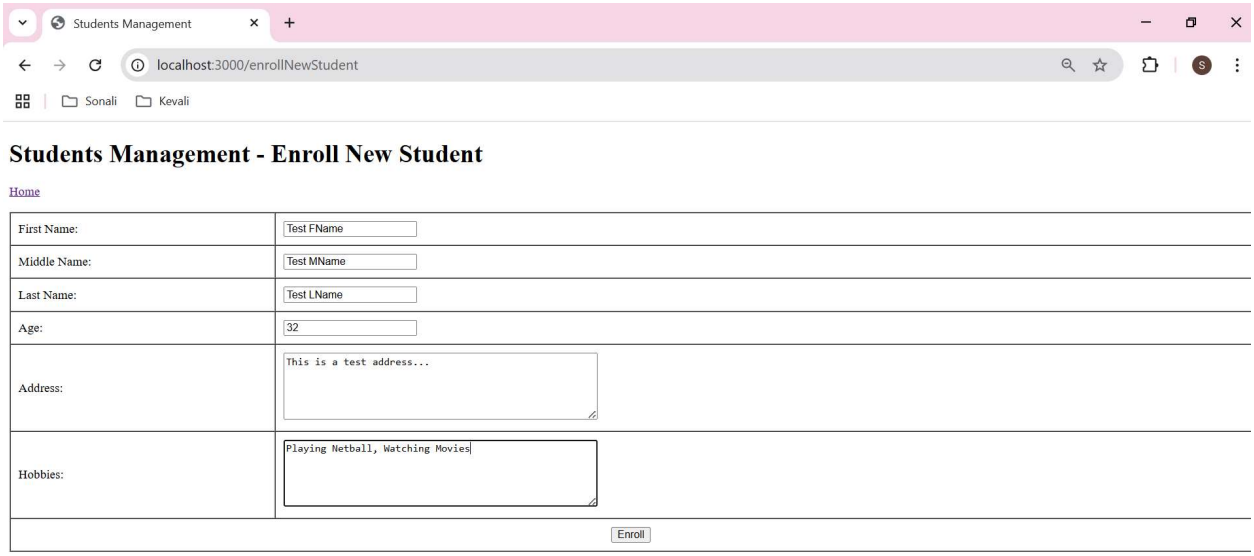
Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > views > viewAllStudents.pug

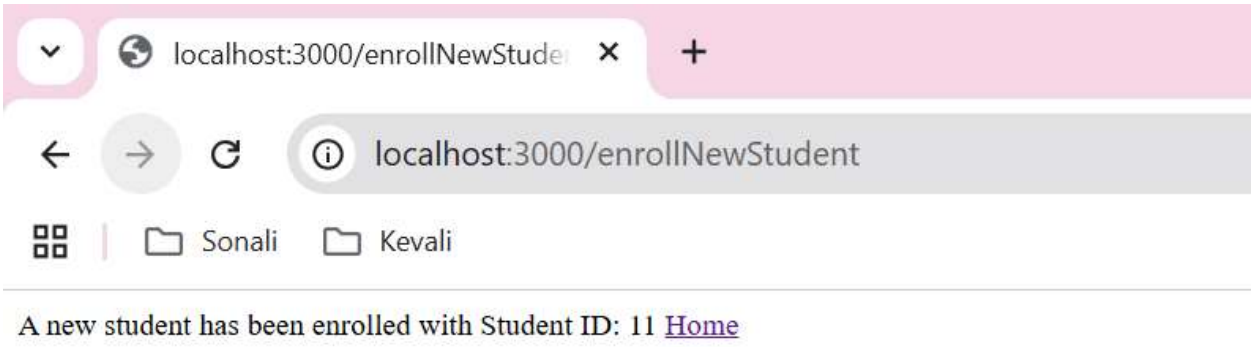
```
html
  head
  body
    h1 Students Management - View All Students
    p
        a(href="/") Home
    p
        table(width="100%" border="1" cellpadding="10" cellspacing="0")
            tbody
                tr
                    th Student ID
                    th First Name
                    th Middle Name
                    th Last Name
                    th Age
                    th(width="400px") Address
                    th Hobbies
                    th Action
                each student in students
                    tr
                        td #{student.student_id}
                        td #{student.first_name}
                        td #{student.middle_name}
                        td #{student.last_name}
                        td #{student.age}
                        td #{student.address}
                        td #{student.hobbies}
                        td
                            a(href=`/updateStudentInfo/${student.student_id}`) Update |
                            a(href=`/deleteStudentRecord/${student.student_id}`) Delete
```

- A GET endpoint ‘/enrollNewStudent’ renders pug view ‘enrollNewStudent.pug’. It displays an empty form for enrolling the student.



- A POST endpoint '/enrollNewStudent' retrieves the data entered by the user in 'enrollNewStudent.pug' template and inserts a new student record in students database table. It provides a link for user to redirect to Home page.





Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
5	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete
6	James	Alexander	Wilson	27	222, Walnut Blvd	Swimming, Playing Chess	Update | Delete
7	Pooja	Rahul	Mehta	2	111, Simei Drive	Programming, Swimming, Reading	Update | Delete
8	Sameer	Abdul	Menon	23	12, Choa Chu Kang Avenue 3	Cricket	Update | Delete
9	Test FName	Test MName	Test LNAme	21	Test Address	Test Hobbies	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete
11	Test FName	Test MName	Test LName	32	This is a test address...	Playing Netball, Watching Movies	Update | Delete

Code from app.js:

```
24
25
26  app.get('/enrollNewStudent', function (req, res) {
27      res.render('enrollNewStudent');
28  });
29
30
31  app.post('/enrollNewStudent', function (req, res) {
32      db.run(`INSERT INTO students (first_name, middle_name, last_name, age, address, hobbies)
33              VALUES (?, ?, ?, ?, ?, ?)`,
34          [req.body.first_name,
35          req.body.middle_name,
36          req.body.last_name,
37          req.body.age,
38          req.body.address,
39          req.body.hobbies], function (err) {
40              if (err) {
41                  console.log(err.message);
42                  res.send('Error enrolling a new student: ' + err.message);
43              }
44              console.log(`A new student has been enrolled with Student ID ${this.lastID}`);
45              res.send('A new student has been enrolled with Student ID: ' + this.lastID + ' <a href="/">Home</a>');
46          });
47  });
48
49
```

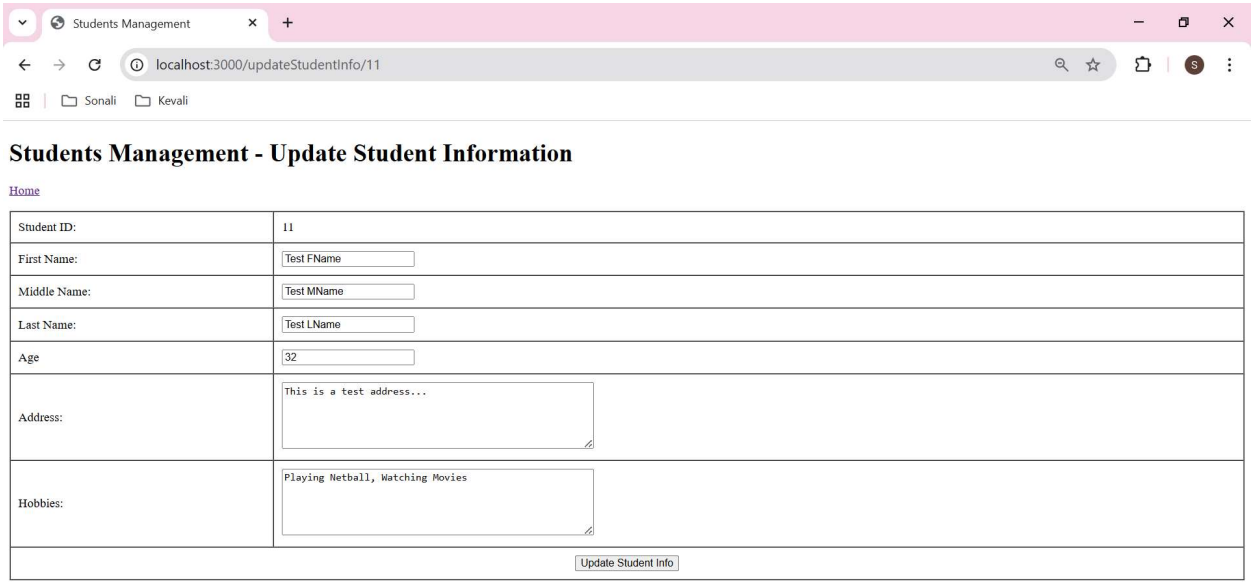
Code from enrollNewStudent.pug:

enrollNewStudent.pug U

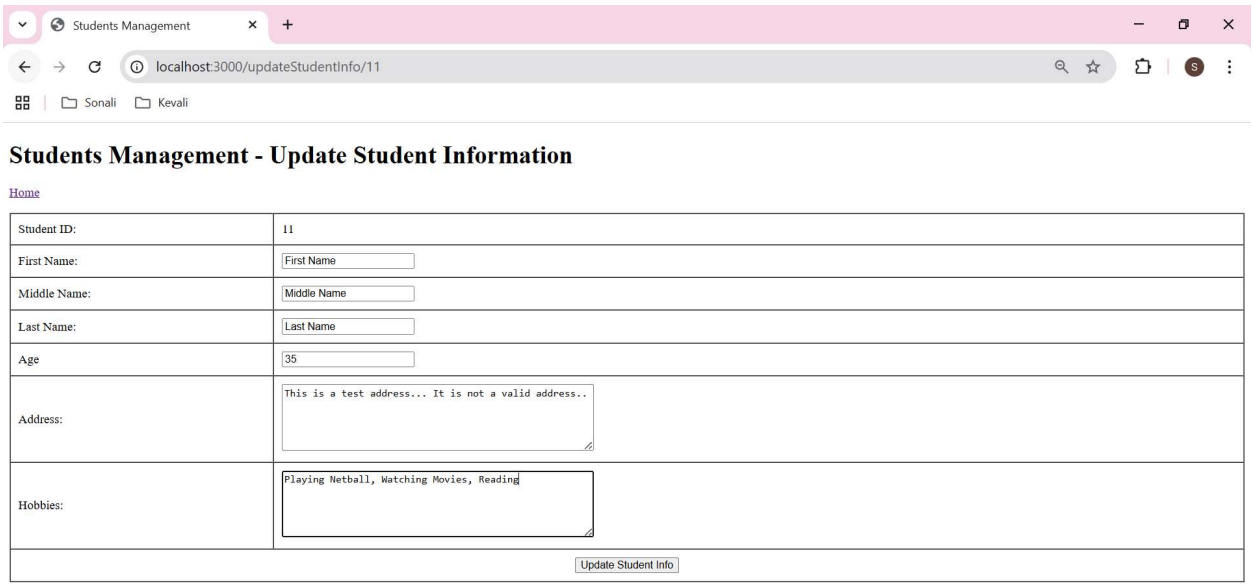
Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > views > enrollNewStudent.pug

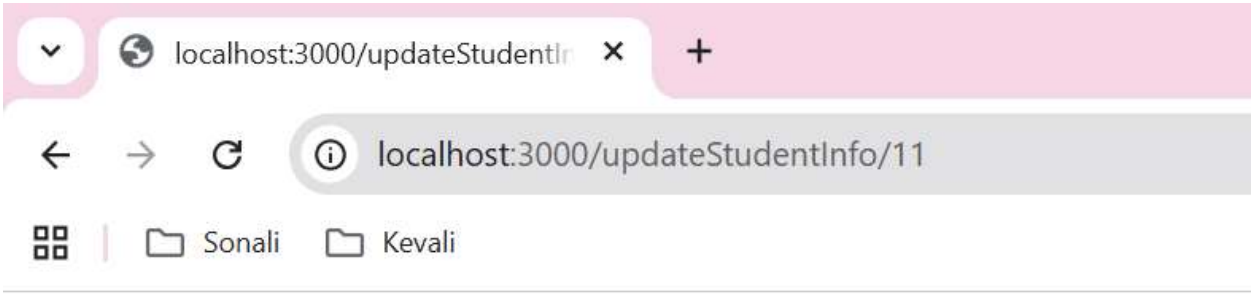
```
doctype html

html
  head
    title Students Management
  body
    h1 Students Management - Enroll New Student
    p
        a(href="/") Home
    p
        form(action="/enrollNewStudent", method="post")
            table(width="100%" border="1" cellpadding="10" cellspacing="0")
                tbody
                    tr
                        td First Name:
                        td
                            input(type="text", name="first_name", required="required")
                    tr
                        td Middle Name:
                        td
                            input(type="text", name="middle_name", required="required")
                    tr
                        td Last Name:
                        td
                           input(type="text", name="last_name", required="required")
                    tr
                        td Age:
                        td
                            input(type="number", name="age", required="required")
                    tr
                        td Address:
                        td
                            textarea(name="address", rows="5", cols="50")
                    tr
                        td Hobbies:
                        td
                            textarea(name="hobbies", rows="5", cols="50")
                    tr
                        td(colspan=2, align="center")
                            input(type="submit", value="Enroll")
```

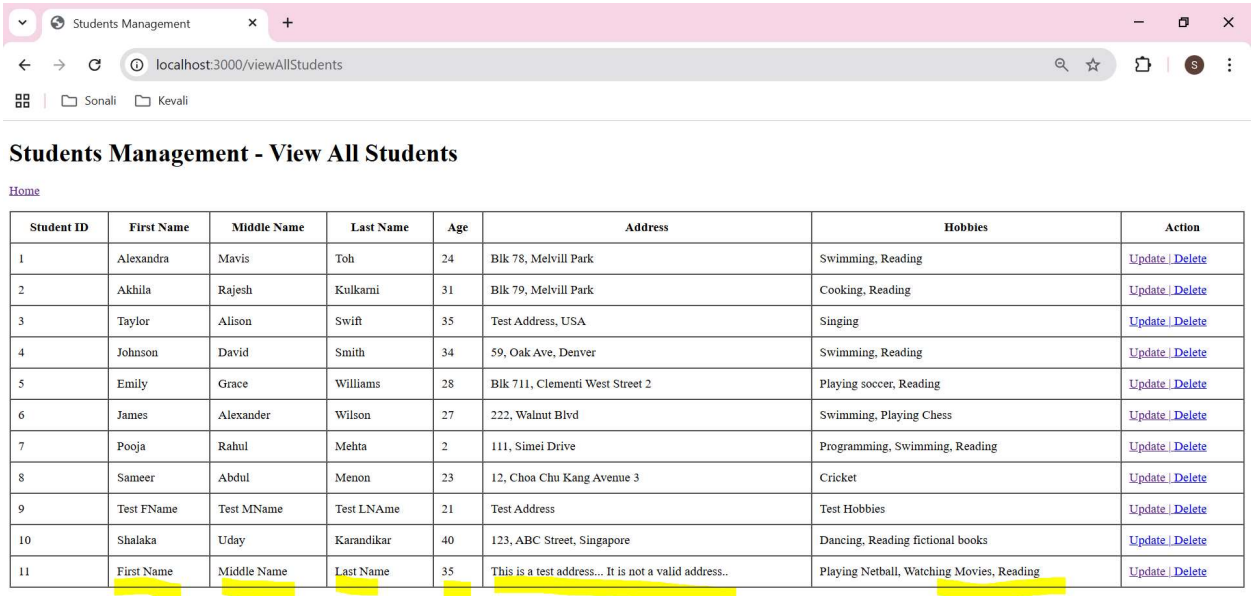
- A GET endpoint '/updateStudentInfo' accepts 'student_id' as parameter and retrieves the record for that particular student from the sqlite database. It then displays that record in pug view, 'enrollNewStudent.pug. User can edit the existing record data here.



- A POST endpoint '/updateStudentInfo' accepts 'student_id' as parameter, retrieves the data edited by user for that particular student id from pug template, 'enrollNewStudent.pug'. It then updates the student record in sqlite students database. Then it provides a link for user to redirect to Home page.







Students Management - View All Students

Home

Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
5	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete
6	James	Alexander	Wilson	27	222, Walnut Blvd	Swimming, Playing Chess	Update | Delete
7	Pooja	Rahul	Mehta	2	111, Simei Drive	Programming, Swimming, Reading	Update | Delete
8	Sameer	Abdul	Menon	23	12, Choa Chu Kang Avenue 3	Cricket	Update | Delete
9	Test FName	Test MName	Test LName	21	Test Address	Test Hobbies	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete
11	First Name	Middle Name	Last Name	35	This is a test address... It is not a valid address..	Playing Netball, Watching Movies, Reading	Update | Delete

Code from app.js:

```
48
49
50  app.get('/updateStudentInfo/:student_id', function (req, res) {
51      db.get('SELECT * FROM students WHERE student_id = ?', [req.params.student_id], (err, row) => {
52          if (err) {
53              console.log(err.message);
54              res.send('Error retrieving student record: ' + err.message);
55          }
56          res.render('updateStudentInfo', { student: row });
57      });
58  });
59
60  app.post('/updateStudentInfo/:student_id', function (req, res) {
61      db.run(`UPDATE students SET first_name = ?, middle_name = ?, last_name = ?, age = ?, address = ?, hobbies = ?  WHERE student_id = ?`,
62          [req.body.first_name,
63          req.body.middle_name,
64          req.body.last_name,
65          req.body.age,
66          req.body.address,
67          req.body.hobbies,
68          req.params.student_id], function (err) {
69              if (err) {
70                  console.log(err.message);
71                  res.send('Error updating student record: ' + err.message);
72              }
73              console.log(`Student with Student ID ${req.params.student_id} is updated`);
74              res.send('Student record updated successfully! <a href="/">Home</a>');
75          });
76  });
77
78
```

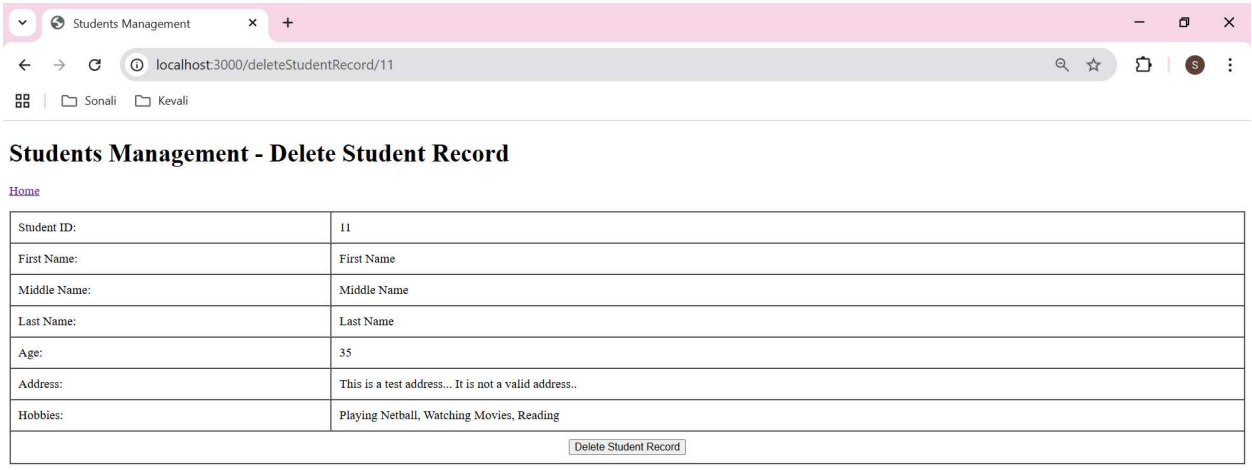
Code from updateStudentInfo.pug:

updateStudentInfo.pug U

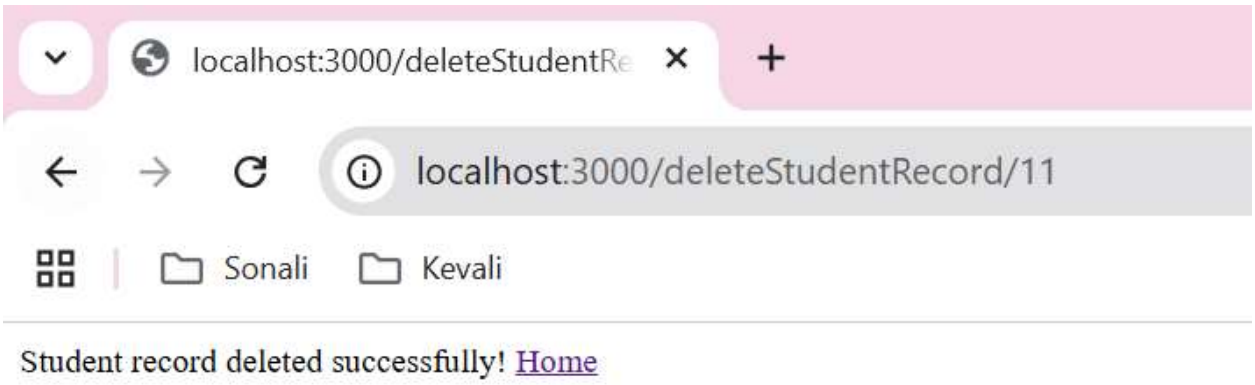
Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > views > updateStudentInfo.pug

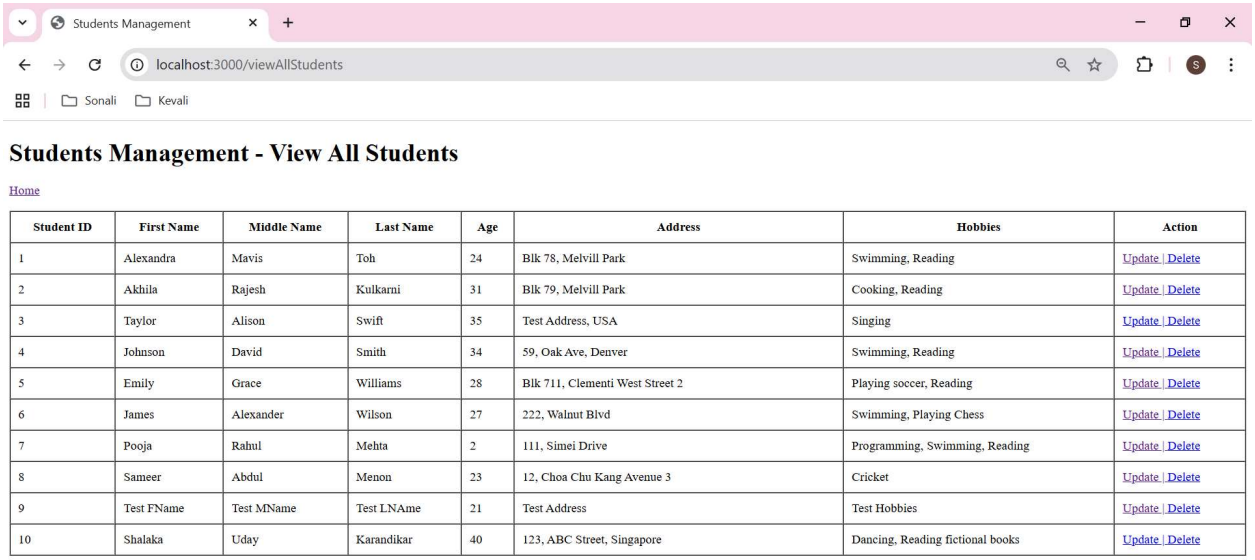
```
doctype html

html
  head
    title Students Management
  body
    h1 Students Management - Update Student Information
    p
        a(href="/") Home
    p
        form(action="/updateStudentInfo/" + student.student_id, method="post")
            table(width="100%" border="1" cellpadding="10" cellspacing="0")
                tbody
                    tr
                        td Student ID:
                        td #{student.student_id}
                    tr
                        td First Name:
                        td
                            input(type="text", name="first_name", required="required", value=student.first_name)
                    tr
                        td Middle Name:
                        td
                            input(type="text", name="middle_name", required="required", value=student.middle_name)
                    tr
                        td Last Name:
                        td
                            input(type="text", name="last_name", required="required", value=student.last_name)
                    tr
                        td Age
                        td
                            input(type="number", name="age", required="required", value=student.age)
                    tr
                        td Address:
                        td
                            textarea(name="address", rows="5", cols="50") #{student.address}
                    tr
                        td Hobbies:
                        td
                            textarea(name="hobbies", rows="5", cols="50") #{student.hobbies}
                    tr
                        td(colspan=2, align="center")
                            input(type="submit", value="Update Student Info")
```

- A GET endpoint '/deleteStudentRecord' accepts 'student_id' as parameter, it then retrieves the record for that particular student from 'students' database table and displays it using 'deleteStudentRecord.pug' pug template.



- A POST endpoint '/deleteStudentRecord' accepts 'student_id' as parameter, it then Deletes the record for that student by its id. Then it provides a link for user to redirect to Home page.





Student ID	First Name	Middle Name	Last Name	Age	Address	Hobbies	Action
1	Alexandra	Mavis	Toh	24	Blk 78, Melvill Park	Swimming, Reading	Update | Delete
2	Akhila	Rajesh	Kulkarni	31	Blk 79, Melvill Park	Cooking, Reading	Update | Delete
3	Taylor	Alison	Swift	35	Test Address, USA	Singing	Update | Delete
4	Johnson	David	Smith	34	59, Oak Ave, Denver	Swimming, Reading	Update | Delete
5	Emily	Grace	Williams	28	Blk 711, Clementi West Street 2	Playing soccer, Reading	Update | Delete
6	James	Alexander	Wilson	27	222, Walnut Blvd	Swimming, Playing Chess	Update | Delete
7	Pooja	Rahul	Mehta	2	111, Simei Drive	Programming, Swimming, Reading	Update | Delete
8	Sameer	Abdul	Menon	23	12, Choa Chu Kang Avenue 3	Cricket	Update | Delete
9	Test FName	Test MName	Test LNAme	21	Test Address	Test Hobbies	Update | Delete
10	Shalaka	Uday	Karandikar	40	123, ABC Street, Singapore	Dancing, Reading fictional books	Update | Delete

Code from app.js:

```
77
78
79  app.get('/deleteStudentRecord/:student_id', function (req, res) {
80      db.get('SELECT * FROM students WHERE student_id = ?', [req.params.student_id], (err, row) => {
81          if (err) {
82              console.log(err.message);
83              res.send('Error retrieving student record: ' + err.message);
84          }
85          res.render('deleteStudentRecord', { student: row });
86      });
87  });
88
89
90  app.post('/deleteStudentRecord/:student_id', function (req, res) {
91      db.run('DELETE FROM students WHERE student_id = ?', [req.params.student_id], function (err) {
92          if (err) {
93              console.log(err.message);
94              res.send('Error deleting student record: ' + err.message);
95          }
96          console.log(`Student with ID ${req.params.student_id} deleted!`);
97          res.send('Student record deleted successfully! <a href="/">Home</a>');
98      });
99  });
100
```

Code from deleteStudentRecord.pug:

deleteStudentRecord.pug U ×

Module 10 > Module 10_Week 12_Required Assignment_Patil > Task 1-4 > views > deleteStudentRecord.pug

```
1   doctype html
2
3   html
4     head
5       title Students Management
6     body
7       h1 Students Management - Delete Student Record
8       p
9           a(href="/") Home
10      p
11          form(action="/deleteStudentRecord/" + student.student_id, method="post")
12              table(width="100%" border="1" cellpadding="10" cellspacing="0")
13                  tbody
14                      tr
15                          td Student ID:
16                          td #{student.student_id}
17                      tr
18                          td First Name:
19                          td #{student.first_name}
20                      tr
21                          td Middle Name:
22                          td #{student.middle_name}
23                      tr
24                          td Last Name:
25                          td #{student.last_name}
26                      tr
27                          td Age:
28                          td #{student.age}
29                      tr
30                          td Address:
31                          td #{student.address}
32                      tr
33                          td Hobbies:
34                          td #{student.hobbies}
35                      tr
36                          td(colspan=2, align="center")
37                              input(type="submit", value="Delete Student Record")
```